Name: _____ Student ID: _____ Date: _____
Name: _____ Student ID: _____

Worksheet #3
Introduction to ARM Processor and Basic Instructions

Objectives

- To understand the architecture and basic operation of ARM processor
- To access (read/write) ARM's registers using assembly language instruction
- To understand the usage and operation of ARM's basic arithmetic instructions

1. Transform the following English sentences to ARM assembly program.

	AREA transform, CODE, READONLY ENTRY Start
1. Put a decimal number "200" to register R0	mov r0,#200 ……
2. Put a hexadecimal number "4F" to register R1	mov r1,#0x4F ……
3. Copy data in register R0 to register R2	mov r2,r0 ……
4. Copy data in register R1 to register R3	mov r3,r1 ……
5. Do R0 + R1 and store result in R4	add r4,r0,r1 ……
6. Do R3 – R1 and store result in R5	sub r5,r3,r1 ……
7. Do R2 + R5 and store result in R2	add r2,r2,r5 ……
8. Do R0 – R1 and store result in R0	sub r0,r0,r1 ……
	Loop B Loop END

What values do the following registers store after executing the above program ?(in hexadecimal)

R0 = 200 ……..
R1 = 79 ……..
R2 = 200 ……..
R3 = 79 ……..
R4 = 279 ……..
R5 = 0 ……..

2. Given the following code, write down the data stored in register R0, R1 and R2 (in hexadecimal) **after** each instruction is executed.

Code	R0	R1	R2

AREA program1, CODE, READONLY
ENTRY
start MOV r0, #20
MOV r1, #-16
MOV r2, #0x52
MOV r0, #-0x52
 	
loop B loop
END
	20
	……
	-82
	……
	……
	……
		-16
		……
		……
		……
		……
			……
			……
			……
			……

Explain for each instruction, how do you get the answers?

```
MOV  r0, #20     ; Load decimal 20 into register r0
MOV  r1, #-16    ; Load decimal -16 into register r1
MOV  r2, #0x52   ; Load hex 0x52 (82 decimal) into register r2
MOV  r0, #-0x52  ; Load negative hex -0x52 (-82 decimal) into r0,
```

3. Given the following code, write down the data stored in register R0, R1 and R2 (in hexadecimal) **after** each instruction is executed.

Code	R0	R1	R2
AREA program1, CODE, READONLY			
ENTRY			
start MOV r0, #20	20	10	96
MOV r1, #10	.....	.....	.....
MOV r2, #0x60	126	116	.....
ADD r2, r2, r0	.....	.....	.....
ADD r0, r1, r2	.....	.....	.....
loop B loop			
END			

Explain for each instruction, how do you have the answers

```
MOV  r0, #20       ; Load decimal 20 into register r0
MOV  r1, #10       ; Load decimal 10 into register r1
MOV  r2, #0x60     ; Load hex 0x60 (96 decimal) into register
ADD  r2, r2, r0    ; Add r2 and r0 (96 + 20 = 116) and store re
ADD  r0, r1, r2    ; Add r1 and r2 (10 + 116 = 126) and store r
```

4. Given the following code, write down the data stored in register R0, R1 and R2 (in hexadecimal) **after** each instruction is executed.

Code	R0	R1	R2
AREA program2, CODE, READONLY			
ENTRY			
start MOV r0, #100	100	20	20
MOV r1, #0x14	……	……	……
MOV r2, r1	……	80	……
SUB r1, r0, r2	……	……	……
SUB r1, r1, r0	……	-20	……
loop B loop			
END			

Explain for each instruction how do you have the answers

```
MOV r0, #100        ; Load the decimal value 100 into register r0
MOV r1, #0x14       ; Load the hex value 0x14 (which is 20 decim
MOV r2, r1          ; Copy the value of r1 (20) into r2
SUB r1, r0, r2      ; Subtract r2 from r0 (100 - 20) and store result
SUB r1, r1, r0      ; Subtract r0 from r1 (80 - 100) and store result
```

5. Write ARM Assembly program to calculate this expression by using only register R0 and R1 (output of the computation will be stored in register R0)

$$R0 = 5A_{16} + 18_{16} - 3E_{16}$$

```
AREA    calculate_1,  CODE,   READONLY
ENTRY

        MOV     r0, #0x5A
        MOV     r1, #0x18
        ADD     r0, r0, r1
        MOV     r1, #0x3E
        SUB     r0, r0, r1
```

Data in R0 after the execution (in hexadecimal) :

34

……

6. Write ARM Assembly program to calculate this expression by using only register R0 and R1 (output of the computation will be stored in register R0)

$$R0 = (125_{10} - 2B_{16}) + 4A_{16} - 250_{10}$$

```
AREA    calculate_2,  CODE,   READONLY
ENTRY


        MOV     r0, #125
        MOV     r1, #0x2B
        SUB     r0, r0, r1
        MOV     r1, #0x4A
        ADD     r0, r0, r1
        MOV     r1, #250
        SUB     r0, r0, r1
```

Data in R0 after the execution (in hexadecimal) :

……0x7E……

7. Write ARM Assembly program to calculate this expression by using only register R0 and R1 (output of the computation will be stored in register R1)

$$R1 = (4F_{16} + 6E_{16}) - 150_{10} - 1F_{16}$$

```
AREA    calculate_3,  CODE,   READONLY
ENTRY

    MOV    r1, #0x4F
    MOV    r0, #0x6E
    ADD    r1, r1, r0
    MOV    r0, #150
    SUB    r1, r1, r0
    MOV    r0, #0x1F
    SUB    r1, r1, r0
```

Data in R1 after the execution (in hexadecimal) :

0x1F

……

8. Complete the following program in ARM Assembly. This program will **swap** between the data stored in register R0 and R1. So in the end, R0 will store 0xBB and R1 will store 0xAA
   - Move immediate number to register directly **<u>is not allow</u>**
   - You can use only register R0, R1 and R2

```
            AREA    swapdata1,  CODE,   READONLY
            ENTRY
start       MOV   R0,   #0xAA
            MOV   R1,   #0xBB

            MOV R2,R0
            MOV R0,R1
            MOV R1,R2




loop        B    loop
            END
```

9. Complete the following program in ARM Assembly. This program will **swap** between the data stored in register R0, R1, and R2. It means that after program is done executing, R0 will store data of R1, R1 will store data of R2 and R2 will store data of R0 (In the exercise, after program is executed : R0 = 0xBB, R1 = 0xCC, and R2 = 0xAA)
   - Move immediate number to register directly **<u>is not allow</u>**
   - You can use only register R0, R1, R2 and R3

```
            AREA    swapdata2,  CODE,   READONLY
            ENTRY
start       MOV   R0,   #0xAA
            MOV   R1,   #0xBB
            MOV   R2,   #0xCC

            MOV    R3, R0
            MOV    R0, R1
            MOV    R1, R2
            MOV    R2, R3




loop        B    loop
            END
```

10. Given that, before running the program, the values in R0-R4 are byte-size ASCII code of upper case English letters. Write an assembly language program that converts a sequence of five ASCII codes in R0-R4 to five ASCII codes of lower case English letters and stores the converted codes in R5-R9. E.g. the result, before and after running the program, is shown below

    (Before) R0 = 0x48 (H), R1 = 0x41 (A), R2 = 0x50 (P), R3 = 0x50 (P), R4 = 0x59 (Y)
    (After) R5 = 0x68 (h), R6 = 0x61 (a), R7 = 0x70 (p), R8 = 0x70 (p), R9 = 0x79 (y)

```
MOV   R0, #0x48
MOV   R1, #0x41
MOV   R2, #0x50
MOV   R3, #0x50
MOV   R4, #0x59
ADD   R5, R0, #0x20
ADD   R6, R1, #0x20
ADD   R7, R2, #0x20
ADD   R8, R3, #0x20
ADD   R9, R4, #0x20
```

11. Write an assembly language program to calculate the following expression

$$3x + x - 2.$$

Suppose that x is stored in R0 and the result in R7.

```
MOV R0,#5
MOV R1, #3
MUL R7, R0, R1
ADD R7, R7, R0
SUB R7, R7, #2
```